Summarize

Chapter 10. Distributed Data Access

Monday, January 3, 12:43



"Now that we've assigned ownership of the expert profile table to the User Management Service," said Sydney, "how should the Ticket Assignment Service get to the expert location and skills data? As I said before, with the number of reads it does to the database, it's really not feasible to make a remote call every time it needs to query the table."

"Can you modify the way the assignment algorithm works so that we can reduce the number of queries it needs?" asked Addison.

"Beats me," replied Sydney. "Taylen's the one who usually maintains those algorithms."

Addison and Sydney met with Taylen to discuss the data access issue and to see if Taylen could modify the expert assignment algorithms to reduce the nimber of database calls to the expert profile table.

"Are you kidding me?" asked Taylen. "There's no way I can rewrite the assignment algorithms to do what you are asking. Absolutely no way at all."

"But our only other option is to make remote calls to the User Management Service every time the assignment algorithm needs expert data," said Addison.

"What?" screamed Taylen. "We can't do that!"

"That what I said as well," said Sydney. "That means we are back to square one again. This distributed architecture stuff is hard. I hate to say this, but I am actually starting to miss the monolithic application. Wait, I know. What if we made messaging calls to the User Maintenance Service instead of using REST?"

"That's the same thing," said Taylen. "I still have to wait for the information to come back, whether we use messaging, REST, or any other remote access protocol. That table simply needs to be in the same data domain as the ticketing tables."

"There's got to be another solution to access data we no longer own," said Addison. "Let me check with Logan."

In most monolithic systems using a single database, developers don't give a second thought to reading database tables. SQL table joins are commonplace, and with a simple query all necessary data can be retrieved in a single database call. However, when data is broken into separate databases or schemas owned by distinct services, data access for read operations starts to become hard.

This chapter describes the various ways services can gain read access to data they don't own—in other words, outside the bounded context of the services needing the data. The four patterns of data access we discuss in this chapter include the Inter-service Communication pattern, Column Schema Replication pattern, Replicated Cache pattern, and the Data Domain pattern.

Each of these data access patterns has its share of advantages and disadvantages. Yes, once again, *trade-offs*. To better describe each of these patterns, we will return to our Wishlist Service and a Catalog Service example from Chapter 9. The Wishlist Service shown in Figure 10-1 maintains a list of items a customer may want to eventually purchase, and contains the customer ID, item ID, and date the item was added in the corresponding Wishlist table. The Catalog Service is responsible for maintaining all of the items the company sells, and includes the item ID, item description, and static product dimension information, such as the weight, height, length, and so on.

In this example, when a request is made from a customer to display in their wish list, both the item ID *and* and the item description (`item_desc`) are returned to the customer. However, the Wishlist Service does not have the item description in its table; that data is owned by the Catalog Service in a tightly formed bounded context providing change control and data ownership. Therefore, the architect must use one of the data access patterns outlined in this chapter to ensure the Wishlist Service can obtain the product descriptions from the Catalog Service.

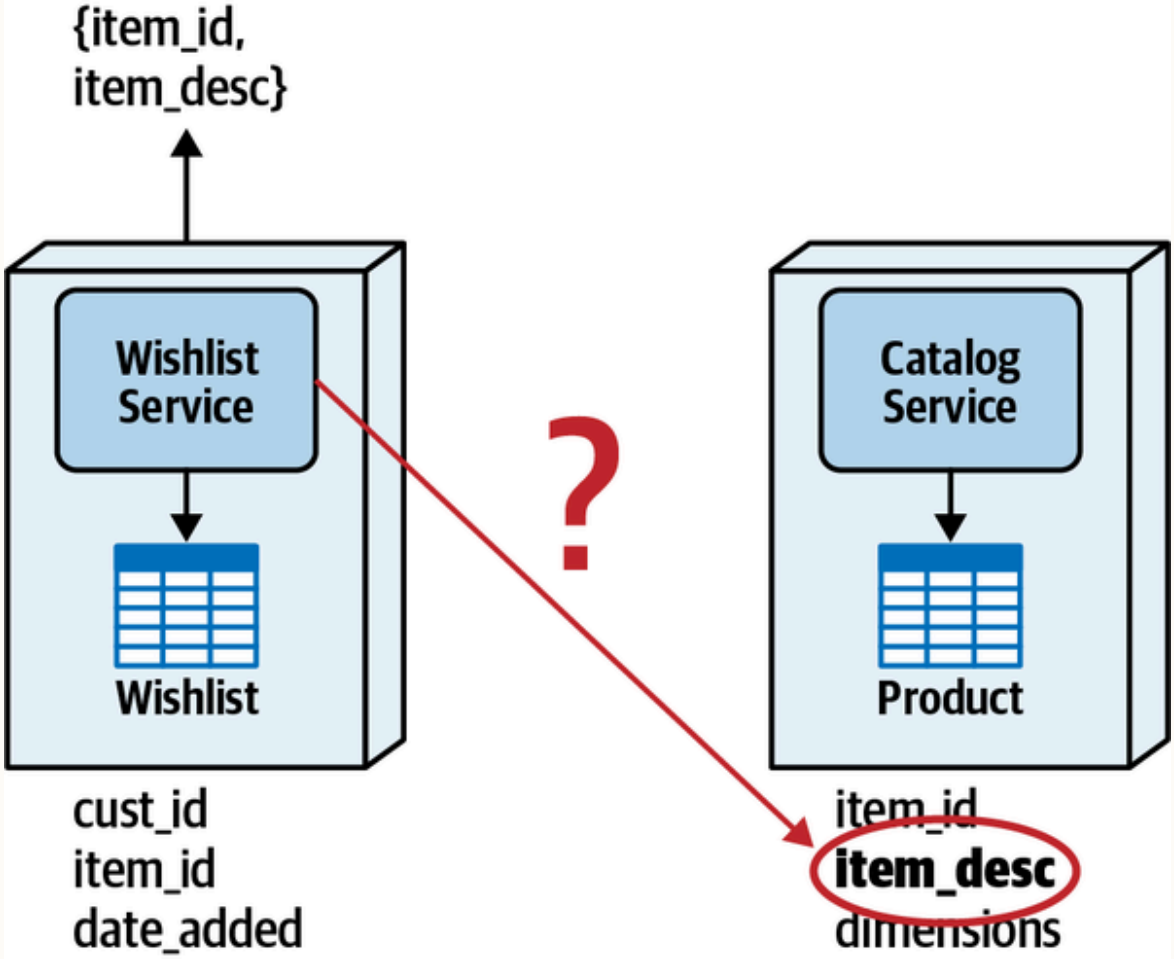


Figure 10-1. Wishlist Service needs item descriptions but doesn't have access to the product table containing the data

Interservice Communication Pattern

The *Interservice Communication pattern* is by far the most common pattern for accessing data in a distributed system. If one service (or system) needs to read data that it cannot access directly, it simply *asks* the owning service or system for it by using some sort of remote access protocol. What can be more simple?

As with most things in software architecture, all is not as it seems. While simple, this common data access technique is unfortunately riddled with disadvantages. Consider Figure 10-2: the Wishlist Service makes a synchronous remote access call to the Catalog Service, passing in a list of item IDs in exchange for a list of corresponding item descriptions.

Notice that for *every* request to get a customer wish list, the Wishlist Service must make a remote call to the Catalog Service to get the item descriptions. The first issue that occurs with this pattern is slower performance due to network latency, security latency, and data latency. *Network latency* is the packet transmission time to and from a service (usually somewhere between 30 ms and 300 ms). Security latency occurs when the endpoint to the target service requires additional authorization to perform the request. *Security latency* can vary greatly depending on the level of security on the endpoint being accessed, but could be anywhere between 20 ms and 400 ms for most systems. *Data latency* describes the situ-

ation where multiple database calls need to be made to retrieve the necessary information to pass back to the end user. In this case, rather than a single SQL table join statement, an additional database call must be made by the Catalog Service to retrieve the item description. This might add anywhere from 10 ms to 50 ms additional processing time. Add all of that up, and the latency could be up to one second just to get the item descriptions.

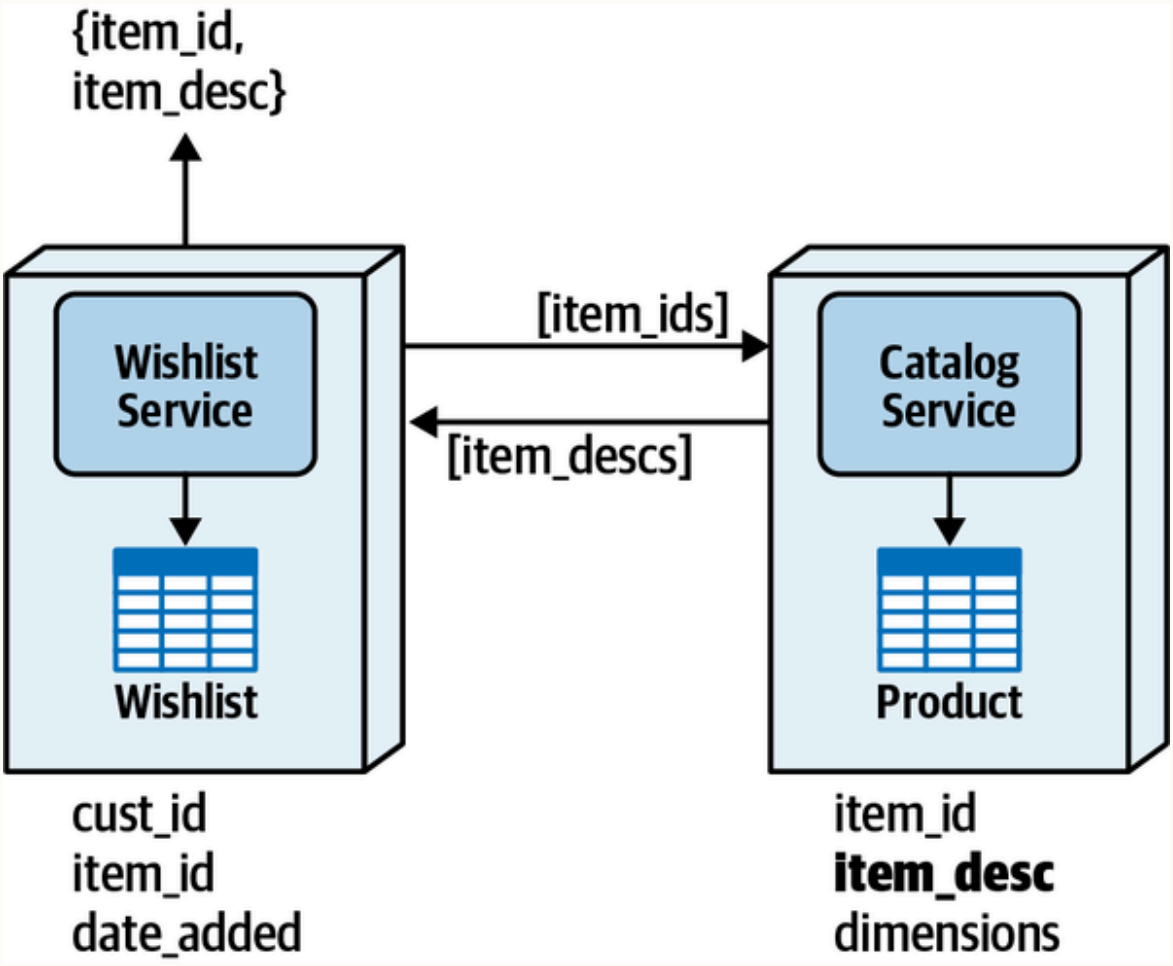


Figure 10-2. Interservice communication data access pattern

Another big disadvantage of this pattern is service coupling. Because the Wishlist must rely on the Catalog Service being available, the services are therefore both semantically and statically coupled, meaning that if the Catalog Service is not available, neither is the Wishlist Service. Furthermore, because of the tight static coupling between the Wishlist Service and the Catalog Service, as the Wishlist Service scales to meet additional demand volume, so must the Catalog Service.

Table 10-1 summarizes the trade-offs associated with the interservice communication data access pattern.

TRADE-OFFS

Table 10-1. Trade-offs for the Interservice Communication data access pattern

Advantages	Disadvantages
Simplicity	Network, data, and security latency (performance)
No data volume issues	Scalability and throughput issues
	No fault tolerance (availability issues)
	Requires contracts between services

Column Schema Replication Pattern

With the *Column Schema Replication pattern*, columns are replicated across tables, therefore replicating the data and making it available to other bounded contexts. As shown in Figure 10-3, the `item_desc` column is added to the Wishlist table, making that data available to the Wishlist Service without having to ask the Catalog Service for the data.

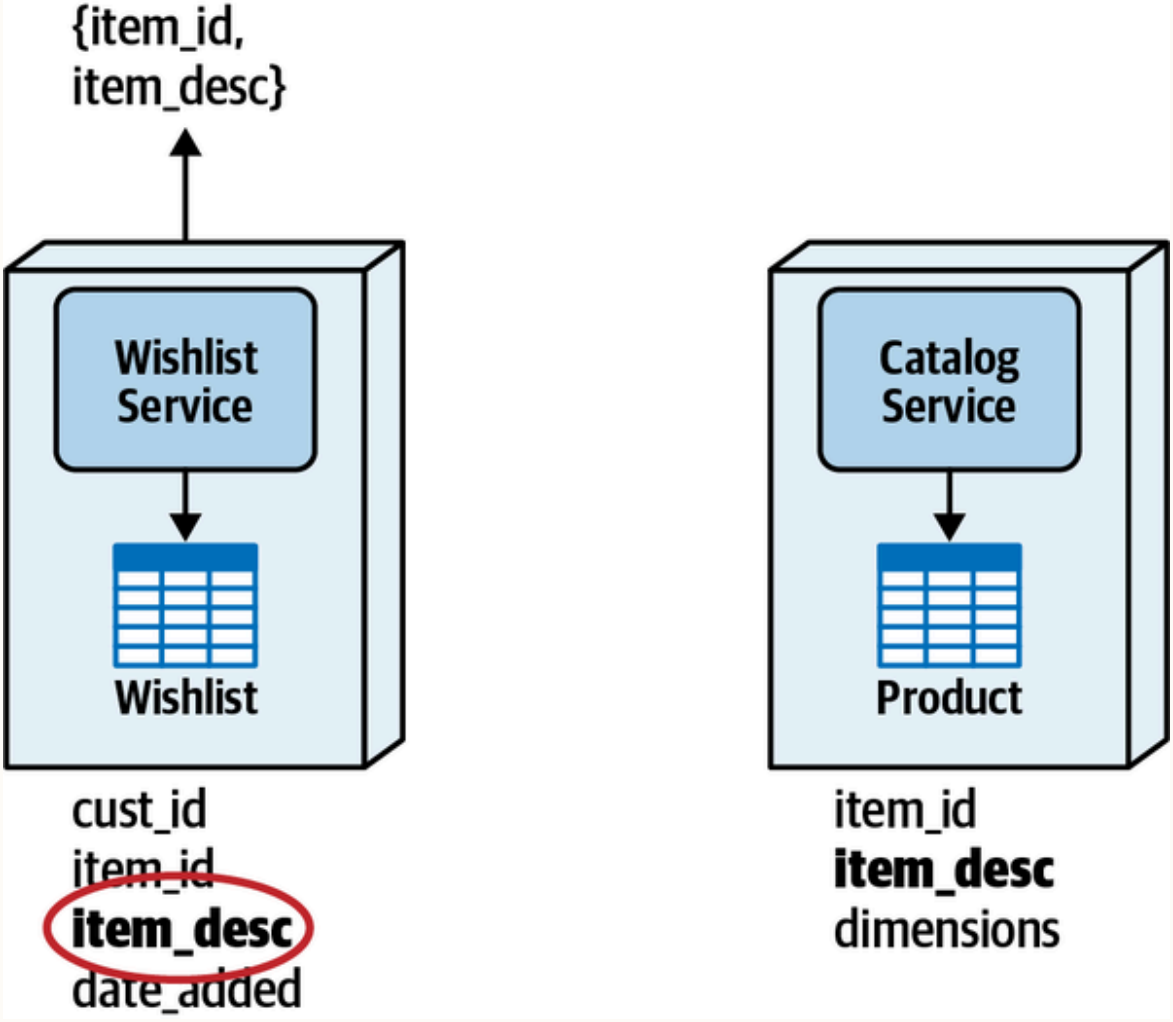


Figure 10-3. With the Column Schema Replication data access pattern, data is replicated to other tables

Data synchronization and data consistency are the two biggest issues associated with the Column Schema Replication data access pattern. Whenever a product is created, removed from the catalog, or a product description changed, the Catalog Service must somehow let the Wishlist Service (and any other services replicating the data) know about the change. This is usually done through asynchronous communications using queues, topics, or event streaming. Unless *immediate* transactional synchronization is required, asynchronous communication is a preferred choice over synchronous communication because it increases responsiveness and reduces the availability dependency between the services.

Another challenge with this pattern is that it is sometimes difficult to govern data ownership. Because the data is replicated in tables belonging to other services, those services can update the data, even though they don't officially *own* the data. This in turn creates even more data consistency issues.

Even though the services are still coupled because of data synchronization, the service requiring read access has immediate access to the data, and can do simple SQL joins or queries to its own table to get the data. This increases performance, fault tolerance, and scalability, all things that were disadvantages with the interservice communication pattern.

While in general we caution against use of this data access pattern for scenarios such as the Wishlist Service and Catalog Service example, some situations where it might be a consideration are data aggregation, reporting, or situations where the other data access patterns are not a good fit because of large data volumes, high responsiveness requirements, or high-fault tolerance requirements.

Table 10-2 summarizes the trade-offs associated with the Column Schema Replication data access pattern.

TRADE-OFFS

Table 10-2. Trade-offs for the Column Schema Replication data access pattern

Advantages	Disadvantages
Good data access performance	Data consistency issues
No scalability and throughput issues	Data ownership issues
No fault-tolerance issues	Data synchronization is required
No service dependencies	

Replicated Caching Pattern

Most developers and architects think of caching as a technique for increasing overall responsiveness. By storing data within an in-memory cache, retrieving data goes from dozens of milliseconds to only a couple of nanoseconds. However, caching can also be an effective tool for distributed data access and sharing. This pattern leverages *replicated in-memory caching* so that data needed by other services is made available to each service without them having to ask for it. A replicated cache differs from other caching models in that data is held in-memory within each service and is continuously synchronized so that all services have the same exact data at all times.

To better understand the replicated caching model, it's useful to compare it to other caching models to see the differences between them. The *single in-memory* caching model is the simplest form of caching, where each service has its own internal in-memory cache. With this caching model (illustrated in Figure 10-4), in-memory data is not synchronized between the caches, meaning each service has its own unique data specific to that service.

While this caching model does help increase responsiveness and scalability within each service, it's not useful for sharing data between services because of the lack of cache synchronization between the services.

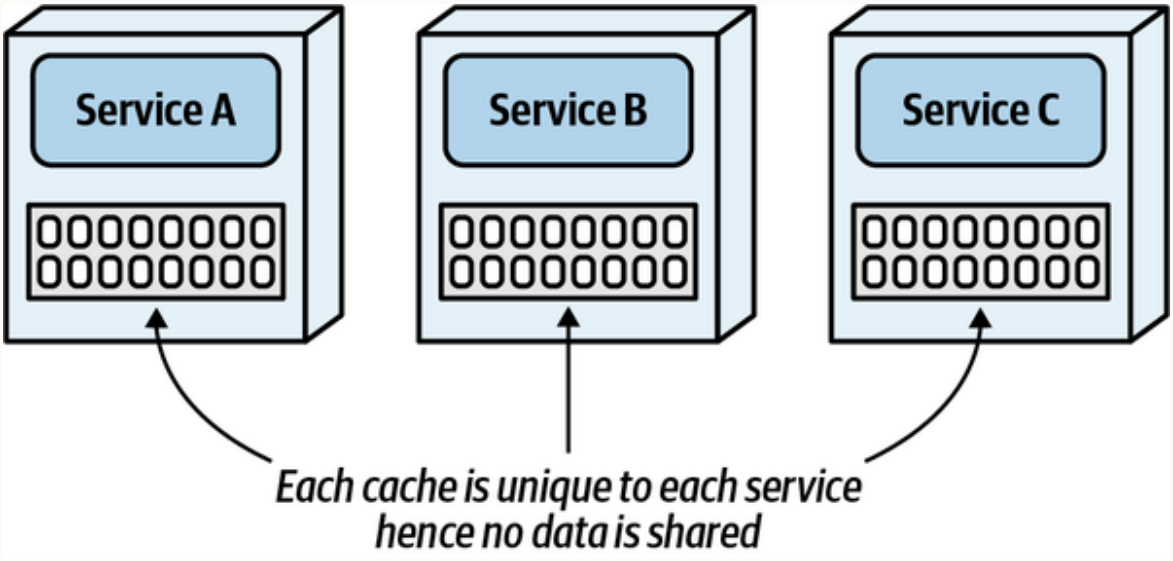


Figure 10-4. With a single in-memory cache, each service contains its own unique data

The other caching model used in distributed architectures is *distributed caching*. As illustrated in Figure 10-5, with this caching model, data is not held in-memory within each service, but rather held externally within a caching server. Services, using a proprietary protocol, make requests to the caching server to retrieve or update shared data. Note that unlike the single in-memory caching model, data can be shared among the services.

The distributed cache model is not an effective caching model to use for the replicated caching data access pattern for several reasons. First, there's no benefit to the fault-tolerance issues found with the Interservice Communication pattern. Rather than depending on a service to retrieve data, the dependency has merely shifted to the caching server.

Because the cache data is centralized and shared, the distributed cache model allows other services to update data, thereby breaking the bounded context regarding data ownership. This can cause data inconsistencies between the cache and the owning database. While this can sometimes be addressed through strict governance, it is nevertheless an issue with this caching model.

Lastly, since access to the centralized distributed cache is through a remote call, network latency adds additional retrieval time for the data, thus impacting overall responsiveness as compared to an in-memory replicated cache.

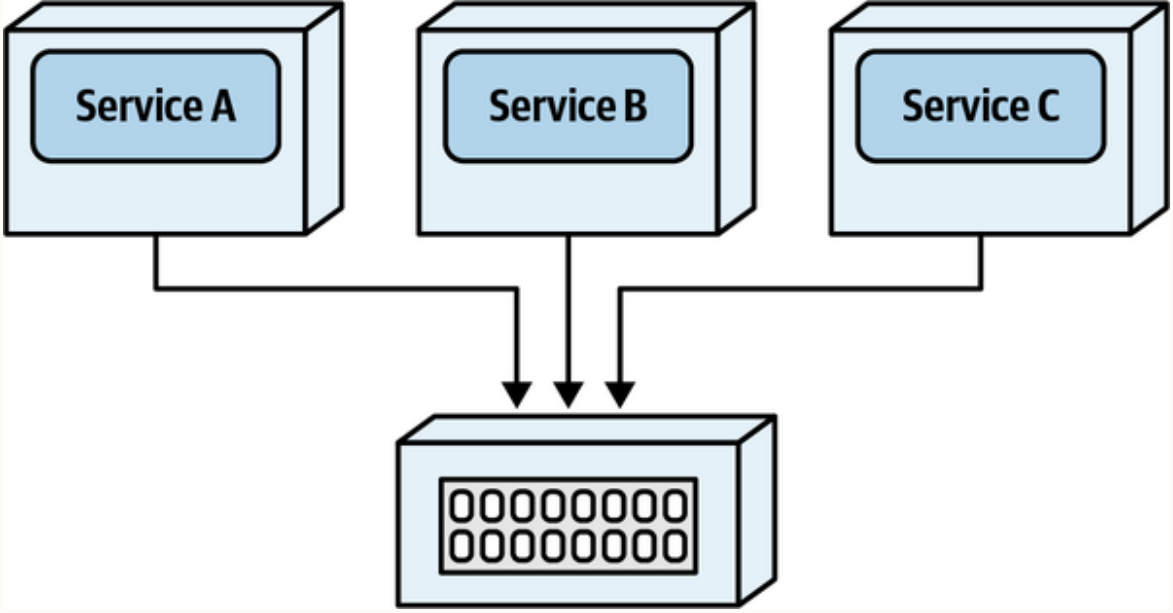


Figure 10-5. A distributed cache is external from the services

With replicated caching, each service has its own in-memory data that is kept in sync between the services, allowing the same data to be shared across multiple services. Notice in Figure 10-6 that there is no external cache dependency. Each cache instance communicates with another so that when an update is made to a cache, that update is immediately (behind the scenes) asynchronously propagated to other services using the same cache.

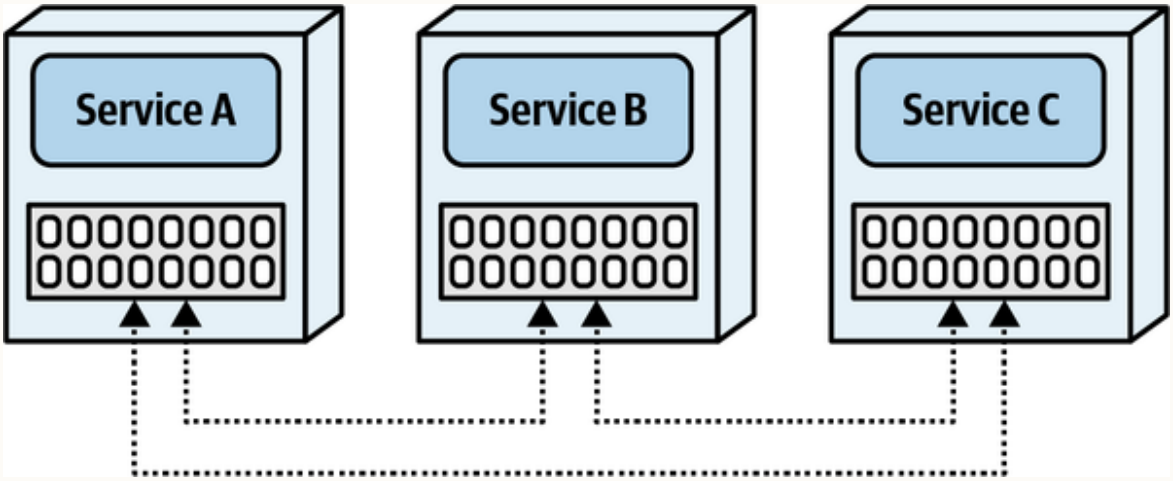


Figure 10-6. With a replicated cache, each service contains the same in-memory data

Not all caching products support replicated caching, so it's important to check with the caching product vendor to ensure support for the replicated caching model. Some of the popular products that do support replicated caching include Hazelcast, Apache Ignite, and Oracle Coherence.

To see how replicated caching can address distributed data access, we'll return to our Wishlist Service and Catalog Service example. In Figure 10-7, the Catalog Service owns an in-memory cache of product descriptions (meaning it is the only service that can modify the cache), and the Wishlist Service contains a read-only in-memory replica of the same cache.

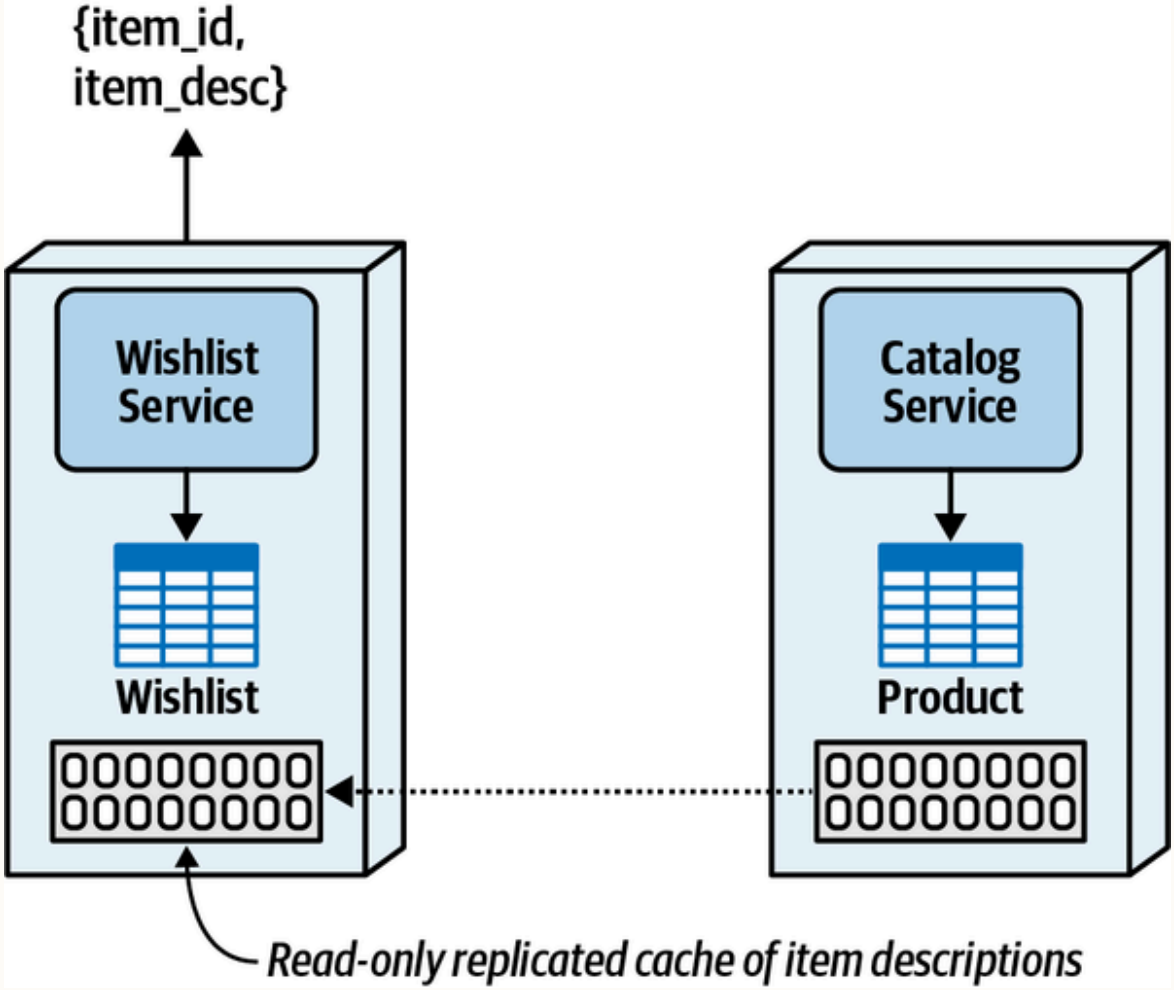


Figure 10-7. Replicated caching data access pattern

With this pattern, the Wishlist Service no longer needs to make calls to the Catalog Service to retrieve product descriptions—they're already in-memory within the Wishlist Service. When updates are made to the product description by the Catalog Service, the caching product will update the cache in the Wishlist Service to make the data consistent.

The clear advantages of the replicated caching pattern are responsiveness, fault tolerance, and scalability. Because no explicit interservice communication is required between the services, data is readily available *in-memory*, providing the fastest possible access to data a service doesn't own. Fault tolerance is also well supported with this pattern. Even if the Catalog Service goes down, the Wishlist Service can continue to operate. Once the Catalog Service comes back up, the caches connect to one another without any disruption to the Wishlist Service. Lastly, with this pattern, the Wishlist Service can scale independently from the Catalog Service.

With all these clear advantages, how could there possibly be a trade-off with this pattern? As the *first law of software architecture* states in our book, *The Fundamentals of Software Architecture*, everything in software architecture is a trade-off, and if an architect thinks they have discovered something that *isn't* a trade-off, it means they just haven't *identified* the trade-off yet.

The first trade-off with this pattern is a service dependency with regard to the cache data and startup timing. Since the Catalog Service owns the cache and is responsible for populating the cache, it must be running when the initial Wishlist Service starts up. If the Catalog Service is unavailable, the initial Wishlist Service must go into a wait state until a connection with the Catalog Service is established. Notice that only the *initial* Wishlist Service instance is impacted by this startup dependency; if the Catalog Service is down, other Wishlist instances can be started up, with the cache data transferred from one of the other Wishlist instances. It's also important to note that once the Wishlist Service starts and has the data in the cache, it is *not* necessary for the Catalog Service to be available. Once the cache is made available in the Wishlist Service, the Catalog Service can come up and down without impacting the Wishlist Service (or any of its instances).

The second trade-off with this pattern is that of data volumes. If the volume of data is too high (such as exceeding 500 MB), the feasibility of this pattern diminishes quickly, particularly with regard to multiple instances of services needing the data. Each service instance has its own replicated cache, meaning that if the cache size of 500 MB and 5 instances of a service are required, the total memory used is 2.5 GB. Architects must analyze both the size of the cache *and* the total number of services instances needing the cached data to determine the total memory requirements for the replicated cache.

A third trade-off is that the replicated caching model usually cannot keep the data fully in sync between services if the rate of change of the data (update rate) is too high. This varies based on the size of the data and the replication latency, but in general this pattern is not well suited for highly volatile data (such as product inventory counts). However, for relatively static data (such as a product description), this pattern works well.

The last trade-off associated with this pattern is that of configuration and setup management. Services know about each other in the replicated caching model through TCP/IP broadcasts and lookups. If the TCI/IP broadcast and lookup range is too broad, it can take a long time to establish the socket-level handshake between services. Cloud-based and containerized environments make this particularly challenging because of the lack of control over IP addresses and the dynamic nature of IP addresses associated with these environments.

Table 10-3 lists the trade-offs associated with the replicated cache data access pattern.

TRADE-OFFS

Table 10-3. Trade-offs associated with the replicated caching data access pattern

Advantages	Disadvantages
Good data access performance	Cloud and containerized configuration can be hard
No scalability and throughput issues	Not good for high data volumes
Good level of fault tolerance	Not good for high update rates
Data remains consistent	Initial service startup dependency
Data ownership is preserved	

Data Domain Pattern

In the previous chapter, we discussed the use of a *data domain* to resolve joint ownership, where multiple services both need to write data to the same table. Tables that are shared between services are put into a single schema that is then shared by both services. That same pattern can be used for data access as well.

Consider the Wishlist Service and Catalog Service problem again, where the Wishlist Service needs access to the product descriptions but does not have access to the table containing those descriptions. Suppose the Interservice Communication pattern is not a feasible solution because of reliability issues with the Catalog Service as well as the performance issues with network latency and the additional data retrieval. Also suppose using the Column Schema Replication pattern is not feasible because of the need for high levels of data consistency. Finally, suppose that the Replicated Cache pattern isn't an option because of the high data volumes. The only other solution is to create a data domain, combining the Wishlist and Product tables in the same shared schema, accessible to both the Wishlist Service and the Catalog Service.

Figure 10-8 illustrates the use of this data access pattern. Notice that the Wishlist and Product tables are no longer owned by either service, but rather shared between them, forming a broader bounded context. With this pattern, gaining access to the product descriptions in the Wishlist Service is a matter of a simple SQL join between the two tables.

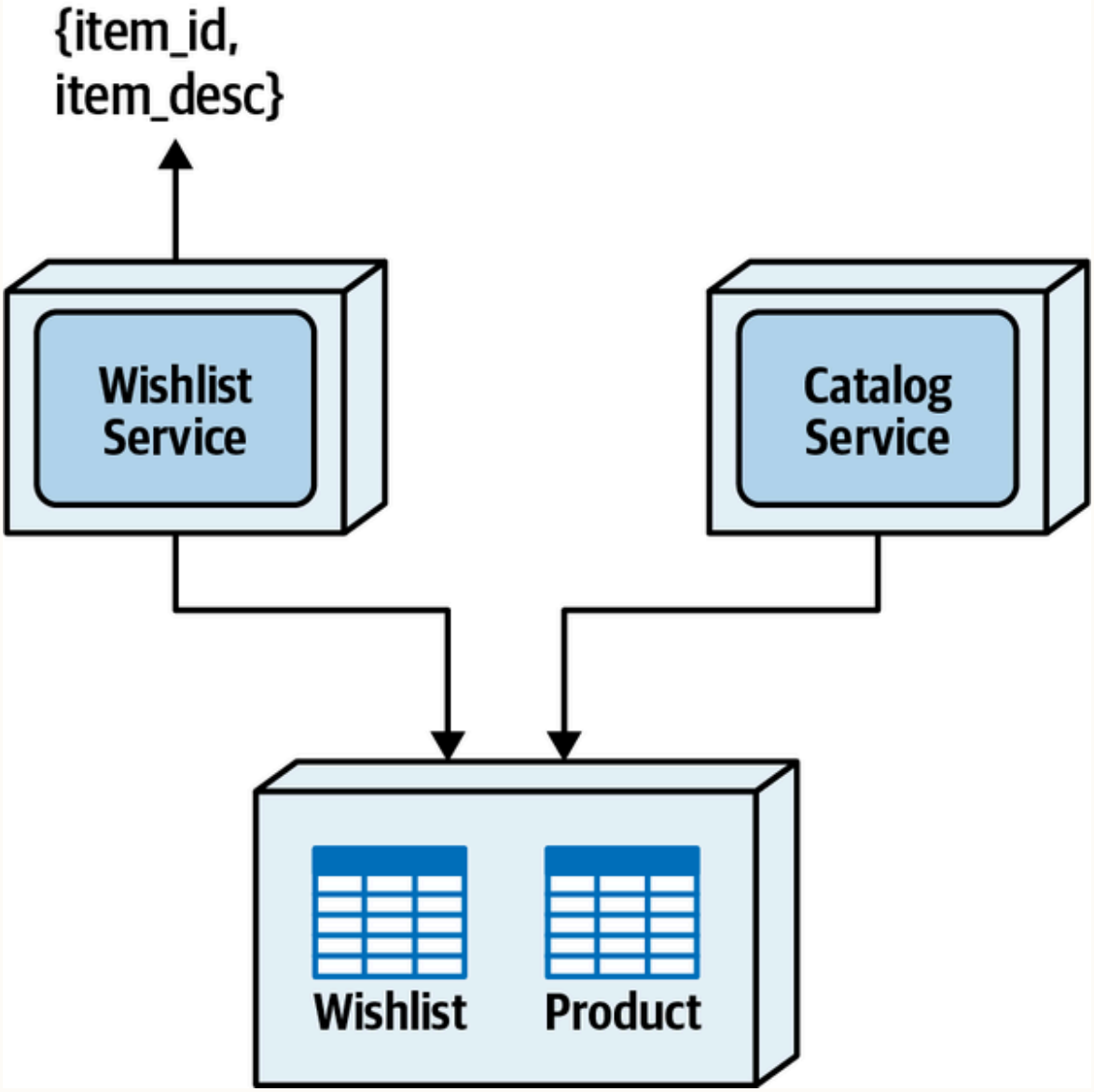


Figure 10-8. Data domain data access pattern

While the sharing of data is generally discouraged in a distributed architecture, this pattern has huge benefits over the other data access patterns. First of all, the services are completely decoupled from each other, thereby resolving any availability dependency, responsiveness, throughput, and scalability issues. Responsiveness is very good with this pattern because the data is available using a normal SQL call, removing the need to do additional data aggregations within the functionality of the service (as is required with the Replicated Cache pattern).

Both data consistency and data integrity rate very high with the Data Domain pattern. Since multiple services access the same data tables, data does not need to be transferred, replicated, or synchronized. Data integrity is preserved in this pattern in the sense that foreign-key constraints can now be enforced between the tables. In addition, other database artifacts, such as views, stored procedures, and triggers, can exist within the data domain. As a matter of fact, the preservation of these integrity constraints and database artifacts is another driver for the use of the Data Domain pattern.

With this pattern, no additional contracts are needed to transfer data between services—the table schema becomes the contract. While this is an advantage for this pattern, it's a trade-

off as well. The contracts used with the interservice communication pattern and the Replicated Cache pattern form an abstraction layer over the table schema, allowing changes to the table structures to remain within a tight bounded context and not impact other services. However, this pattern forms a broader bounded context, requiring multiple services to possibly change when the structure to any of the tables in the data domain changes.

Another disadvantage of this pattern is that it can possibly open up security issues associated with data access. For example, in Figure 10-8 the Wishlist Service has complete access to *all* the data within the data domain. While this is OK in the Wishlist and Catalog Service example, there might be times when services accessing the data domain shouldn't have access to certain data. A tighter bounded context with strict service ownership can prevent other services from accessing certain data through the contracts used to pass the data back and forth.

Table 10-4 lists trade-offs associated with the data domain data access pattern.

TRADE-OFFS

Table 10-4. Trade-offs associated with the data domain data access pattern

Advantages	Disadvantages
Good data access performance	Broader bounded context to manage data changes
No scalability and throughput issues	Data ownership governance
No fault tolerance issues	Data access security
No service dependency	
Data remains consistent	

Sysops Squad Saga: Data Access for Ticket Assignment

Thursday, March 3, 14:59



Logan explained the various methods for data access within a distributed architecture, and also outlined the corresponding trade-offs of each technique. Addison, Sydney, and Taylen then had to come to a decision about which technique to use.

“Unless we start consolidating all of these services, I guess we are stuck with the fact that the Ticket Assignment needs to somehow get to the expert profile data, and fast,” said Taylen.

“OK,” said Addison. “So service consolidation is out because these services are in entirely different domains, and the shared data domain option is out for the same reasons we talked about before—we cannot have the Ticket Assignment Service connecting to two different databases.”

“So, that leaves us with one of two choices.” said Sydney. “Either we use interservice communication or replicated caching.”

“Wait. Let's explore the replicated caching option for a minute,” said Taylen. “How much data are we talking about here?”

“Well,” said Sydney, “we have 900 experts in the database. What data does the Ticket Assignment Service need from the expert profile table?”

“It’s mostly static information as we get the current expert location feeds from elsewhere. So, it would be the expert's skill, their service location zones, and their standard scheduled availability,” said Taylen.

“OK, so that’s about 1.3 KB of data per expert. And since we have 900 experts total, that would be...about 1200 KB of data total. And the data is relatively static,” said Sydney.

“Hmm, that isn’t much data to store in memory,” said Taylen.

“Let’s not forget that if we used a replicated cache, we would have to take into account how many instances we would have for the User Management Service as well as the Ticket Assignment Service,” said Addison. “Just to be on the safe side, we should use the maximum number of instances of each we expect.”

“I’ve got that information,” said Taylen. “We expect to have only a maximum of two instances of the User Management Service, and a maximum of four at our highest peak for the Ticket Assignment Service.”

“That’s not much total in-memory data,” observed Sydney.

“No, it’s not,” said Addison. “OK, let’s analyze the trade-offs using the hypothesis-based approach we tried earlier. I suggest that we should go with the in-memory replicated cache option to cache only the data necessary for the Ticket Assignment Service. Any other trade-offs you can think of?”

Both Taylen and Sydney sat there for while trying to think of some negatives for the replicated cache approach.

“What if the User Management Service goes down?” asked Sydney.

“As long as the cache is populated, then the Ticket Assignment Service would be fine,” said Addison.

“Wait, you mean to tell me that the data would be in-memory, even if the User Management Service is unavailable?” asked Taylen.

“As long as the User Management Service starts before the Ticket Assignment Service, then yes,” said Addison.

“Ah!” said Taylen. “Then there’s our first trade-off. Ticket assignment cannot function unless the User Management Service is started. That’s not good.”

“But,” said Addison, “if we made remote calls to the User Management Service and it goes down, the Ticket Assignment Service becomes nonoperational. At least with the replicated cache option, once User Management is up and running, we are no longer dependent on it. So, replicated caching is actually more fault tolerant in this case.”

“True,” said Taylen. “We just have to be careful about the startup dependency.”

“Anything else you can think of as a negative?” asked Addison, knowing another obvious trade-off but wanting the development team to come up with it on their own.

“Um,” said Sydney, “yeah. I have one. What caching product are we going to use?”

“Ah,” said Addison, “that is in fact another trade-off. Have either of you done replicated caching before? Or anyone on the development team for that matter?”

Both Taylen and Sydney shook their heads.

“Then we have some risk here,” said Addison.

“Actually,” said Taylen, “I’ve been hearing a lot about this caching technique for a while and have been dying to try it out. I would volunteer to research some of the products and do

some proof-of-concepts on this approach."

"Great," said Addison. "In the meantime, I will research what the licensing cost would be for those products as well, and if there's any technical limitation with respect to our deployment environment. You know, things like availability zone crossovers, firewalls, that sort of stuff."

The team began their research and proof-of-concept work, and found that this is indeed not only a feasible solution cost and effort wise, but would solve the issue of data access to the expert profile table. Addison discussed this approach with Logan, who approved the solution. Addison created an ADR outlining and justifying this decision.

> *ADR: Use of In-Memory Replicated Caching for Expert Profile Data*
>
> *Context*
> *The Ticket Assignment Service needs continuous access to the expert profile table, which is owned by the User Management Service in a separate bounded context. Access to the expert profile information can be done through interservice communication, in-memory replicated caching, or a common data domain.*
>
> *Decision*
> *We will use replicated caching between the User Management Service and the Ticket Assignment Service, with the User Management Service being the sole owner for write operations.*
>
> *Because the Ticket Assignment Service already connects to the shared ticket data domain schema, it cannot connect to an additional schema. In addition, since the user management functionality and the core ticketing functionality are in two separate domains, we do not want to combine the data tables in a single schema. Therefore, using a common data domain is not an option.*
>
> *Using an in-memory replicated cache resolves the performance and fault-tolerance issues associated with the interservice communication option.*
>
> *Consequences*
> *At least one instance of the User Management Service must be running when starting the first instance of the Ticket Assignment Service.*
>
> *Licensing costs for the caching product would be required for this option.*

Previous chapter

< 9. Data Ownership and Distributed Transactions

Next chapter

11. Managing Distributed Workflows >